Breaking the Memory and Communication Walls of Distributed Complex GEMM on Multi-Zone Heterogeneous DSP Supercomputers

Anonymous Author(s)
Design Automation Conference (DAC) Blind Review Submission

***Abstract*—Double-precision complex general matrix-matrix multiplication (ZGEMM) is foundational to computational electromagnetics, quantum chemistry, and dense linear algebra solvers in electronic design automation (EDA). Deploying large-scale ZGEMM on multi-zone heterogeneous supercomputers—exemplified by the MT-3000 processor coupling 16 ARMv8 cores with four autonomous 24-core bare-metal VLIW/SIMD DSP clusters per node—reveals two primary constraints: on-chip scratchpad bandwidth saturation without hardware caches, and inter-node collective communication serialization across thousands of accelerator processes.**

We present an end-to-end hardware-software co-design framework addressing both memory and network bottlenecks. At the intra-chip level, we architect a 14-channel asynchronous DMA tri-buffered scratchpad pipeline (C-matrix cyclic triple buffering nested around A/B ping-pong double buffering) paired with a 4×4 register-resident dual-FMA micro-kernel that pins 32 physical 16-lane FP64 vector accumulators, eliminating 86.1% ($7.20\times$) of Array Memory vector traffic. At the inter-node level, we formulate a Stationary-A, Panel-Broadcast-B 2D hybrid orchestration ($P/2 \times 2$) incorporating zero-copy strided panel extraction via committed `MPI_Type_vector` datatypes, two-phase non-blocking subgroup collectives, offset-addressed parallel POSIX `pread` directly into DSP-mapped memory overlapped with network transfers, and 64-column vector-aligned intra-cluster partitioning. On single-node benchmarks ($N = 512$ to 16384), 1 DSP core sustains 49.19 GFLOPS (64.1% of dual-FMA peak; $6.12\times$–$7.28\times$ over OpenBLAS), 1 cluster reaches 243.44 GFLOPS ($\sim 100\%$ of the 240.5 GFLOPS DDR4 DMA Roofline; $9.33\times$ over 4-core CPU), and 1 full node reaches 956.17 GFLOPS with FP64 ℓ_∞ error $\leq 2.18 \times 10^{-8}$. Scaling across 512 MT-3000 nodes (2,048 MPI processes, 49,152 DSP cores), our co-design reduces 8192^3 collective time by $9.90\times$ ($102.25 \rightarrow 10.32$ s), halves the DDR-streaming floor to 89.92 TFLOPS ($2.01\times$ over 1D), scales extreme 16384^3 workloads to 218.79 TFLOPS ($222.49\times$ speedup, 112.80% 2-node superlinear efficiency), and maintains 92.21% weak-scaling efficiency at 49,152 DSP cores.

***Index Terms*—Complex GEMM (ZGEMM), multi-zone processor, MT-3000 DSP, scratchpad memory (SPM), asynchronous DMA triple buffering, 2D hybrid MPI collectives, Roofline model.**

I. Introduction

Double-precision complex general matrix-matrix multiplication (ZGEMM), defined for split-complex matrices $A \in \mathbb{C}^{M\times K}$, $B \in \mathbb{C}^{K\times N}$, and $C \in \mathbb{C}^{M\times N}$ as $C \leftarrow C + AB$, is a foundational computational building block in high-frequency computational electromagnetics, quantum chemistry, radar imaging, and complex matrix solvers in electronic design automation (EDA) [1], [2]. Unlike real DGEMM ($2MNK$ FLOPs), ZGEMM requires $8MNK$ double-precision operations—four real matrix multiplications and four real matrix additions/subtractions—imposing quadrupled arithmetic density and doubled memory footprints.

To address power and energy walls, modern high-performance processors increasingly substitute hardware cache hierarchies with software-managed scratchpad memories (SPMs) and explicit direct memory access (DMA) engines [3]–[6]. The **MT-3000** multi-zone processor [7] exemplifies this architectural paradigm, coupling a 16-core ARMv8 General-Purpose (GP) Zone with a 96-core Acceleration (ACC) Zone partitioned into four autonomous 24-core bare-metal VLIW/SIMD DSP clusters (11.6 TFLOPS node peak at 1.2 GHz). Each DSP core integrates a 64 KB Scalar Memory (SM) and a 768 KB vector Array Memory (AM), with no hardware cache to off-chip DDR4 [8], [9].

While prior work has accelerated real-valued kernels on single MT-3000 nodes [8]–[11], scaling distributed complex ZGEMM across hundreds of heterogeneous nodes exposes four coupled challenges:

1) **Complex Register Pressure and AM Traffic Amplification:** Updating real (C_{Re}) and imaginary (C_{Im}) tiles involves four coupled cross-terms. Naive inner-loop accumulation from AM (2,304 vector memory ops per $4 \times 64 \times 32$ block) starves dual-issue fused multiply-add (FMA) units, whereas algebraic 3M reductions [2] demand auxiliary memory and induce $\mathcal{O}(\kappa)$ numerical cancellation [12].
2) **Asymmetric Scratchpad Capacity and Exposed C-Tile DMA:** All active A, B, and C buffers must co-reside inside 64 KB SM and 768 KB AM. Standard A/B ping-pong double buffering hides inner K-loop fetches but leaves bidirectional DDR4↔AM load and write-back latencies of C exposed at outer tile boundaries.
3) **Supercomputer-Scale Collective Contention and Serialized Root I/O:** Binding one MPI rank per autonomous DSP cluster creates 2,048 ranks across 512 nodes. Globally broadcasting a 1.074 GB complex B matrix ($8192^2 \times 16$ B) incurs severe interconnect contention (102.25 s at 512 nodes—over $1{,}040\times$ the kernel compute time), while root-serialized storage reads leave 2,047 clusters idle.
4) **Strong-Scaling Off-Chip DDR-Streaming Floor:** Under

strong scaling ($N = 8192$), once per-cluster row height M_{local} drops below scratchpad tile height $M_b = 56$ ($P > 146$ in 1D), arithmetic intensity collapses from 14.83 to 1.88 FLOPs/Byte, locking 1D execution to a hard DDR4 streaming floor (0.0982 s, 44.78 TFLOPS).

To overcome these constraints simultaneously, we present a full-stack hardware-software co-design:

- **14-Channel Tri-Buffered Pipeline and 4×4 Micro-Kernel:** We coordinate 14 hardware P2P DMA channels to nest inner A/B ping-pong double buffering (56 KB SM + 64 KB AM) inside middle C-matrix cyclic triple buffering (168 KB AM). Our 4×4 micro-kernel pins 32 physical 16-lane FP64 vector registers across $K_b = 32$ steps using hoisted SPU negation and `vec_svbcast`, cutting AM traffic by 86.1% (7.20×).
- **Stationary-A, Panel-Broadcast-B 2D Orchestration ($P/2 \times 2$):** We eliminate network transfers of A and post-compute reductions of C via disjoint 2D output ownership, zero-copy strided panel dispatch via committed `MPI_Type_vector` types, two-phase non-blocking subgroup collectives overlapped with concurrent POSIX `pread`, and 64-column vector-aligned intra-cluster partitioning, analytically proving the optimality of $G = 2$.
- **Capacity Proof and Strong-Scaling DDR-Floor Theorem:** We establish a closed-form scratchpad capacity model proving that tile $(56, 32, 64)$ achieves $\mathcal{I}_{\text{DMA}} = 14.83$ FLOPs/Byte (240.5 GFLOPS cluster DDR ceiling), and derive the strong-scaling streaming floor theorem predicting a 2.0× asymptotic kernel speedup at scale.
- **Multi-Scale 1- to 512-Node Supercomputer Validation:** Our kernel achieves 49.19 GFLOPS/core (64.1% peak; 6.12×–7.28× over OpenBLAS), 243.44 GFLOPS/cluster (∼ 100% Roofline), and 956.17 GFLOPS/node. Across 512 nodes (49,152 DSP cores), our co-design cuts 8192^3 collective time by 9.90×, doubles kernel throughput to 89.92 TFLOPS (2.01× over 1D), reaches 218.79 TFLOPS at 16384^3 (222.49× speedup, 112.80% 2-node superlinear efficiency), and sustains 92.21% weak-scaling efficiency.

II. Architectural Background & Machine Balance

A. MT-3000 Multi-Zone Architecture and Scratchpads

As illustrated in Fig. 1 and detailed in Table I, the MT-3000 processor adopts a heterogeneous multi-zone SoC architecture [7]. The *GP Zone* comprises 16 ARMv8 CPU cores running Linux and the MPI runtime. The *ACC Zone* integrates four physically autonomous DSP clusters managed via the lightweight `Hthread` driver. Because each DSP cluster maintains independent command queues, on-chip shared memories, and DMA controllers without inter-cluster cache coherence, we bind one MPI rank exclusively to each DSP cluster ($\text{dev_id} = \text{rank} \bmod 4$, 4 ranks/node), eliminating intra-node device contention.

Each DSP cluster houses 24 bare-metal DSP cores clocked at 1.2 GHz. Within each core, a Scalar Processing Unit (SPU) accesses private 64 KB *Scalar Memory (SM)* and broadcasts

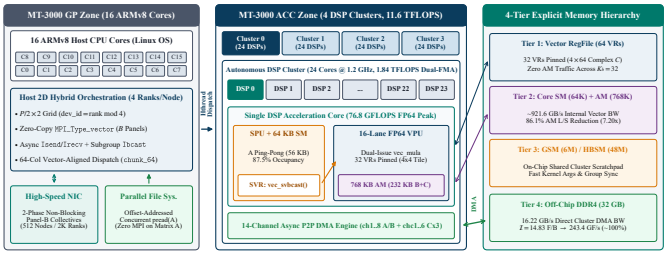


Fig. 1. MT-3000 multi-zone heterogeneous node architecture, 4-tier SPM hierarchy, and mapping of our ZGEMM co-design across ARMv8 and DSP clusters.

TABLE I
Verified Specifications of an MT-3000 Node [7], [9]

Zone / Tier	Parameter	Verified Specification
GP Zone	Host CPU Cores	16 ARMv8 Cores (Linux OS; 4 Ranks/Node)
ACC Zone	DSP Clusters / Cores	4 Clusters/Node (1/Rank); 24 Cores/Cluster (96C/Node)
	Clock Frequency	1.2 GHz; 1,536 FP64 VPU Lanes/Node
	Compute Peak	76.8 GF/s per Core; 1.84 TF/s per Cluster; 11.6 TF/s Node
	DDR4 DMA BW	32 GB Shared DDR4; **16.22 GB/s** Cluster Direct DMA
4-Tier Memory	Vector Regs (VR)	64 × 1,024-bit (16× FP64) per Core
	Scalar Mem (SM)	64 KB per Core (SPU control + `vec_svbcast`)
	Array Mem (AM)	768 KB per Core (∼921.6 GB/s wide vector access)

scalar operands via `vec_svbcast`. A coupled 16-lane FP64 Vector Processing Unit (VPU) features 64 physical 1,024-bit vector registers (`lvector double`) and dual-issue FMA pipelines (`vec_mula`), delivering a theoretical vector peak:

$$\mathcal{P}_{\text{core}} = 16 \times 2\,\text{FMA} \times 2\,\text{FLOPs} \times 1.2\,\text{GHz} = 76.8\,\text{GFLOPS}, \tag{1}$$

yielding $\mathcal{P}_{\text{cluster}} = 1{,}843.2$ GFLOPS per 24-core cluster. The VPU executes wide vector loads/stores (`vec_ld`/`vec_st`) exclusively against its private 768 KB *Array Memory (AM)* at ∼ 921.6 GB/s. Because no hardware cache exists between off-chip DDR4 and SM/AM, all data movement must be explicitly scheduled in software via point-to-point hardware DMA channels (`dma_p2p`), with measured direct cluster DMA bandwidth $\mathcal{B}_{\text{DDR}} = 16.22$ GB/s [9].

B. Split-Complex Planar Format vs. Algebraic 3M

For split-complex matrices $A = A_{\text{Re}} + iA_{\text{Im}}$, $B = B_{\text{Re}} + iB_{\text{Im}}$, and $C = C_{\text{Re}} + iC_{\text{Im}}$, ZGEMM computes:

$$\begin{aligned} C_{\text{Re}} &\leftarrow C_{\text{Re}} + A_{\text{Re}}B_{\text{Re}} - A_{\text{Im}}B_{\text{Im}}, \\ C_{\text{Im}} &\leftarrow C_{\text{Im}} + A_{\text{Re}}B_{\text{Im}} + A_{\text{Im}}B_{\text{Re}}. \end{aligned} \tag{2}$$

Standard Array-of-Structures (AoS) complex layouts ($[x_0, y_0, \ldots]$) force costly in-register shuffling on 16-lane VPUs lacking de-interleaving crossbars. We adopt *split-complex planar* (SoA) layout, aligning 16 consecutive real or imaginary elements directly to 128-byte `vec_ld` operations. Furthermore, while Gauss 3M methods reduce operations to $6MNK$ FLOPs [2], they consume auxiliary DDR4 buffers and suffer $\mathcal{O}(\kappa)$ numerical cancellation [12]. We execute exact 4M accumulation via native dual-issue `vec_mula` instructions, preserving FP64 accuracy without workspace allocation.

C. Machine Balance and the Scratchpad Memory Wall

Dividing cluster peak $\mathcal{P}_{\text{cluster}} = 1{,}843.2$ GFLOPS by off-chip bandwidth $\mathcal{B}_{\text{DDR}} = 16.22$ GB/s exposes a steep machine

balance ridge point [13]:

$$\mathcal{I}_{\text{ridge}} = \mathcal{P}_{\text{cluster}}/\mathcal{B}_{\text{DDR}} = 1{,}843.2/16.22 = 113.64\,\text{FLOPs/B}. \tag{3}$$

For a complex tile (M_b, K_b, N_b) in 64 KB SM and 768 KB AM, transferring $16(M_bK_b + K_bN_b)$ bytes to perform $8M_bK_bN_b$ FLOPs bounds arithmetic intensity to $\mathcal{I}_{\max} < \frac{M_bN_b}{2(M_b+N_b)}$. Even if A occupied the entire 64 KB SM ($M_b \approx 64$), $\mathcal{I}_{\max} \leq 16$ FLOPs/B $\ll 113.64$ FLOPs/B. Thus, *multi-core ZGEMM is fundamentally DDR4-DMA-bound at the cluster level, yet AM-port- and FMA-latency-bound within each core*. Maximizing performance requires saturating tile arithmetic intensity while fully overlapping DDR4 DMA with VPU execution.

III. Tri-Buffered Scratchpad Pipeline & Micro-Kernel

A. 14-Channel Asynchronous DMA Tri-Buffered State Machine

On cacheless DSP cores, conventional A/B ping-pong double buffering leaves the bidirectional DDR4 read and write-back latencies of each $M_b \times N_b$ complex C tile (112 KB round-trip) unhidden between K-reductions. We resolve this via a **14-channel two-tier DMA state machine** (`kernel_pipeline.c`; Fig. 2(a) and Algorithm 1):

1) **Middle-Tier C Cyclic Triple Buffering (6 Channels, `chc1..chc6`):** Three split-complex buffer pairs reside in AM (168 KB total). Cyclic rotation maps buffer $(j/N_b) \bmod 3$ to prefetch $C^{(j)}$ via strided DMA, buffer $(j/N_b - 1) \bmod 3$ to active VPU K-accumulation, and buffer $(j/N_b - 2) \bmod 3$ to asynchronous DDR4 write-back of $C^{(j-2N_b)}$, hiding 100% of bidirectional C DMA traffic behind compute.
2) **Inner-Tier A/B Ping-Pong Double Buffering (8 Channels, `ch1..ch8`):** Two complex A pairs reside in 64 KB SM (`ch1..ch4`, 56 KB); two complex B pairs reside in 768 KB AM (`ch5..ch8`, 64 KB). Non-blocking prefetches for step $k + K_b$ are posted before compute on step k, hoisting all `dma_wait` barriers outside the inner $K_b = 32$ FMA loop.

B. 4×4 Vector-Register-Resident Dual-FMA Micro-Kernel

Naive inner-loop accumulation from AM issues 2,304 vector memory operations for 2,048 FMAs (0.89 FMAs/access), causing severe port contention. We design a 4×4 **outer-product micro-kernel** (Fig. 2(b)):

- **32-VR Accumulator Pinning (Zero Spilling):** We unroll 4 rows along M_b and 4 vector segments ($4 \times 16 = 64$ columns) along N_b, pinning $4 \times 4 \times 2 = \mathbf{32}$ physical vector registers $(\mathbf{cx}_{u,v}, \mathbf{cy}_{u,v})$ for the C sub-tile. With 8 streaming B registers and 3 broadcast A registers, total live register count is 43 of 64 VRs (67.2%), guaranteeing zero compiler spilling to AM.
- **Hoisted SPU Negation for Dual-Issue `vec_mula`:** For each row, the SPU computes $-a_i$ in scalar arithmetic and broadcasts $\{a_r, a_i, -a_i\}$ via `vec_svbcast`, enabling all 32 real and 32 imaginary updates per step to execute exclusively as fused dual-issue `vec_mula` instructions without subtract instructions:

$$\begin{aligned}
\mathbf{cx}_{u,v} &\leftarrow \texttt{vec_mula}(\tilde{\mathbf{a}}_{r,u},\, \mathbf{bx}_v,\, \mathbf{cx}_{u,v}),\\
\mathbf{cx}_{u,v} &\leftarrow \texttt{vec_mula}(-\tilde{\mathbf{a}}_{i,u},\, \mathbf{by}_v,\, \mathbf{cx}_{u,v}),\\
\mathbf{cy}_{u,v} &\leftarrow \texttt{vec_mula}(\tilde{\mathbf{a}}_{r,u},\, \mathbf{by}_v,\, \mathbf{cy}_{u,v}),\\
\mathbf{cy}_{u,v} &\leftarrow \texttt{vec_mula}(\tilde{\mathbf{a}}_{i,u},\, \mathbf{bx}_v,\, \mathbf{cy}_{u,v}).
\end{aligned} \tag{4}$$

- 86.1% **AM Traffic Elimination:** Vectors $\mathbf{cx}_{u,v}, \mathbf{cy}_{u,v}$ are loaded once before $p = 0$ and stored once after $p = 31$. Total AM instructions drop from 2,304 to $32+256+32 = \mathbf{320}$ (7.20× fewer accesses), raising arithmetic density to $64/8 = \mathbf{8.0}$ vector FMAs per vector load.

Algorithm 1 Device-Side 14-Channel Tri-Buffered DSP Kernel

Input: $A \in \mathbb{C}^{M_r \times K}, B \in \mathbb{C}^{K \times N_g}, C \in \mathbb{C}^{M_r \times N_g}$; core $t \in [0, 23]$; $(M_b, K_b, N_b) = (56, 32, 64)$.
Output: Updated core slice $C[:, j_0(t) : j_0(t) + N_{\text{thread}}(t)]$.

Allocate SM ping-pong $A_{1,2}$ (56 KB), AM ping-pong $B_{1,2}$ (64 KB), AM triple $C_{1..3}$ (168 KB).
for $i = 0$ **to** $M_r - 1$ **step** M_b **do**
  Async `dma_p2p` load $C[i, j_0] \to C_1$.
  for $j = N_b$ **to** $N_{\text{thread}}(t) - 1$ **step** N_b **do**
    Rotate $(s_{\text{store}}, s_{\text{comp}}, s_{\text{load}}) \bmod 3$; post async prefetch $C[i, j_0 + j] \to C_{s_{\text{load}}}$.
    Post async $A[i, 0] \to A_1$, $B[0, j_0 + j - N_b] \to B_1$; wait on $C_{s_{\text{comp}}}$ and prior $C_{s_{\text{store}}}$.
    for $k_{\text{idx}} = 0$ **to** $\lceil K/K_b \rceil - 1$ **do**
      $b \leftarrow (k_{\text{idx}} \bmod 2) + 1$; $\bar{b} \leftarrow 3 - b$.
      if $k_{\text{idx}} + 1 < \lceil K/K_b \rceil$ **then**
        Post async prefetch $A_{k+K_b} \to A_{\bar{b}}$ and $B_{k+K_b} \to B_{\bar{b}}$.
      end if
      `dma_wait` on (A_b, B_b); load 32 VRs $\{\mathbf{cx}_{u,v}, \mathbf{cy}_{u,v}\}$ from $C_{s_{\text{comp}}}$.
      for $p = 0$ **to** $K_b - 1$ **do**
        Load 8 vectors B from B_b; broadcast $\{a_r, a_i, -a_i\}$; execute dual `vec_mula`.
      end for
      Store 32 VRs to $C_{s_{\text{comp}}}$.
    end for
    Post async `dma_p2p` store $C_{s_{\text{comp}}} \to C[i, j_0 + j - N_b]$.
  end for
  Finalize remaining tile accumulation and await DMA write-back completion.
end for

C. Scratchpad Capacity Proof and Strong-Scaling Streaming Floor

Selecting (M_b, K_b, N_b) is constrained by SM (64 KB) and AM (768 KB):

$$\begin{aligned}
\mathcal{S}_{\text{SM}} &= 4M_bK_b \times 8\,\text{B} \leq 56\,\text{KB},\\
\mathcal{S}_{\text{AM}} &= (4K_bN_b + 6M_bN_b) \times 8\,\text{B} \leq 768\,\text{KB}.
\end{aligned} \tag{5}$$

Fixing $N_b = 64$ and $K_b = 32$, (5) yields optimal $M_b = 56$ ($56 = 14 \times 4$), using $\mathcal{S}_{\text{SM}} = \mathbf{56.0}$ KB (87.5%) and $\mathcal{S}_{\text{AM}} = \mathbf{232.0}$ KB (30.2%). For $K = 8192$, this yields operational arithmetic intensity:

$$\mathcal{I}_{\text{DMA}} = \frac{8M_bKN_b}{16K(M_b + N_b) + 32M_bN_b} = \mathbf{14.83}\,\text{FLOPs/B}, \tag{6}$$

defining theoretical 24-core cluster Roofline ceiling $\mathcal{P}_{\text{roof}}^{\text{cluster}} = 16.22 \times 14.83 = \mathbf{240.5}$ GFLOPS.

Under strong scaling (P ranks, G panels), $M_{\text{local}} = GM/P$. Once $P > GM/M_b$, $M_{\text{eff}} = M_{\text{local}} < 56$, causing intensity to collapse to $\mathcal{I}_{\text{eff}} \approx \frac{M_{\text{local}}N_b}{2(M_{\text{local}}+N_b)}$. At 512 nodes

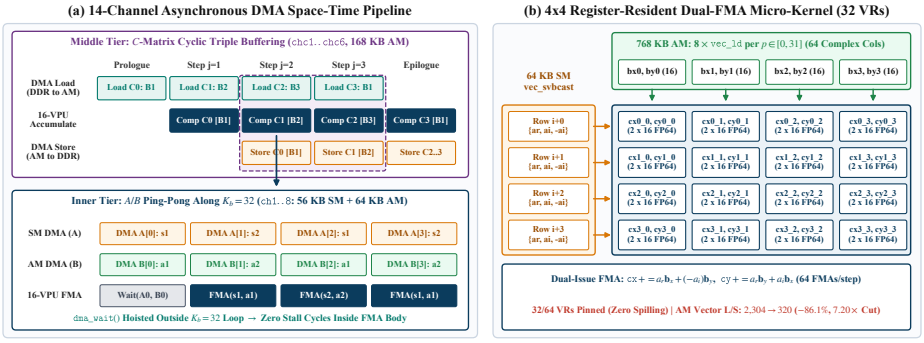


Fig. 2. Intra-chip scratchpad and register co-design: (a) 14-channel asynchronous DMA space-time pipeline nesting inner A/B double buffering inside middle C cyclic triple buffering; (b) 4×4 register-resident dual-FMA micro-kernel pinning 32 vector accumulators to cut AM traffic by 86.1%.

TABLE II
ASYMPTOTIC PER-RANK COMPLEXITY ACROSS DISTRIBUTED GEMM SCHEMES

Distributed Scheme	Network Volume	Sync Stages	Local Rows	DDR-Streaming Floor
1D Flat Broadcast	$16N^2 \log_2 P$	2 Global	N/P	$\geq 16N^2/\mathcal{B}_{\text{DDR}}$
2D SUMMA ($\sqrt{P}$) [14]	$\frac{32N^2}{\sqrt{P}} \log_2 \sqrt{P}$	$\frac{2N}{K_b} \log_2 \sqrt{P}$	$N/\sqrt{P}$	Stalls on Iterative Launches
2.5D (c copies) [15]	$\mathcal{O}(N^2/\sqrt{Pc})$	Bcast + Reduce	$N\sqrt{c/P}$	Extra C-Reduction Traffic
Proposed 2D ($P/G \times G$)	$\frac{16N^2}{G} \log_2(\mathbf{P/G})$	**1 Subgroup**	$\mathbf{GN/P}$	$\geq \frac{16N^2}{G \cdot \mathcal{B}_{\text{DDR}}}$ ($G\times$ **Lower**)

($P = 2{,}048$), 1D ($G = 1, M_{\text{local}} = 4$) collapses to $\mathcal{I}_{\text{eff}} = \mathbf{1.88}$ FLOPs/B, whereas 2D ($G = 2, M_{\text{local}} = 8$) maintains $\mathcal{I}_{\text{eff}} = \mathbf{3.56}$ FLOPs/B. Because each cluster must stream B_g from DDR4, kernel runtime obeys the **DDR-streaming lower bound**:

$$T_{\text{kernel}}(P, G) \geq \frac{16K(N/G) + 16M_{\text{local}}(K + 2N/G)}{\mathcal{B}_{\text{DDR}}}. \quad (7)$$

As $P \to \infty$, $T_{\text{stream}}(1)/T_{\text{stream}}(2) \to \mathbf{2.00}\times$, predicting the $2.01\times$ speedup measured at 512 nodes.

IV. HIERARCHICAL 2D DATA ORCHESTRATION

A. Stationary-A, Panel-Broadcast-B Logical Grid

A 1D broadcast baseline (`gemm_host_AS.c`) broadcasts 100% of B across all ranks, triggering network storms at 512 nodes (102.25 s). General $\sqrt{P} \times \sqrt{P}$ SUMMA [14] incurs $\sqrt{P}$ iterative barriers, while 2.5D replication [15] requires post-compute C reductions and exceeds node RAM.

We formulate a $(P/G) \times G$ **Stationary-A, Panel-Broadcast-B 2D grid** (Fig. 3(a)). With $G = 2$, rank $p \in [0, P-1]$ maps to subgroup $g = p \bmod G$, intra-group index $r = \lfloor p/G \rfloor$, and cluster $d = p \bmod 4$. Matrix B is partitioned into G vertical panels $B_g \in \mathbb{C}^{K \times N_g}$, while A is partitioned into $P_g = P/G$ horizontal slices $A_r \in \mathbb{C}^{M_r \times K}$. Leader ranks ($r = 0$) employ asymmetric load balancing: $M_0 = \lfloor M/P_g \rfloor$, shielding leaders from remainder rows. Each rank exclusively updates disjoint output $C_{r,g} \leftarrow C_{r,g} + A_r B_g$, requiring **zero network communication of A and zero post-compute reduction of C**.

B. Zero-Copy Strided Vector Dispatch and Parallel `pread`

To avoid copying B_g into intermediate buffers at Rank 0, we commit a zero-copy strided datatype (`operator.c`):

$$\mathcal{D}_{B_g} \triangleq \texttt{MPI_Type_vector}(K,\ N_g,\ N,\ \texttt{MPI_DOUBLE}), \quad (8)$$

dispatching B_g via non-blocking `MPI_Isend` to subgroup leaders ($r = 0, g \geq 1$), which unpack into contiguous DSP buffers via `MPI_Irecv` (Phase 1). Leaders then execute non-blocking `MPI_Ibcast` across isolated subgroup communicators $\mathcal{C}_g$ of size P/G (Phase 2).

Concurrently, every rank (r, g) invokes POSIX `pread64` at byte offset $\Delta_r = 8 \cdot i_0(r) \cdot K$ to stream its A_r slice directly from storage into Hthread DMA-mapped memory, completely overlapping file I/O with Phase-1 network progress (Fig. 3(b)).

C. 64-Column Vector Alignment and Optimality of $G = 2$

Distributing N_g columns across $T = 24$ DSP cores requires enforcing **64-column vector alignment** (`chunk_64 = 64`): core t receives $N_{\text{thread}}(t) = 64(q+1)$ for $t < u$, and $64q$ otherwise ($Q = \lfloor N_g/64 \rfloor, q = \lfloor Q/24 \rfloor, u = Q \bmod 24$). For $N = 8192$: (i) 24-core occupancy requires $N_g \geq 24 \times 64 = 1{,}536 \implies G \leq 4$; (ii) 3-stage C-buffering requires $N_{\text{thread}} \geq 3N_b = 192$ columns, ruling out $G = 4$ where 16 cores receive only 64 columns; (iii) $G = 2$ ($N_g = 4{,}096$) provides 192 columns to 16 cores and 128 to 8 cores with zero tail padding, uniquely satisfying all constraints.

V. EXPERIMENTAL EVALUATION

We evaluate our framework on the MT-3000 supercomputer scaling from 1 to 512 nodes (up to 2,048 MPI ranks and 49,152 bare-metal DSP cores, 5.94 PFLOPS aggregate peak). Host code is compiled with `mpicc -O2 -fopenmp`; DSP code is built with the vendor VLIW/SIMD C compiler.

A. Intra-Node Performance, Roofline, and Accuracy

Table III and Fig. 4 summarize intra-node evaluation:

- **Single-Core Throughput:** 1 DSP core sustains 44.16–49.19 GFLOPS (64.1% of peak), outperforming 1-core OpenBLAS by 6.12×–7.28× (`4x DGEMM`) and 5.95×–6.47× (`1x ZGEMM`).
- **Cluster-Level Saturation:** 1 DSP cluster (24 cores) reaches 243.44 **GFLOPS** at $N = 8192$, matching our 240.5 GFLOPS DDR4 Roofline ceiling within 1.2% ($\sim$100% saturation). This outperforms 4-core CPU OpenBLAS by 9.33× and full 16-core CPU OpenBLAS by 2.72×. 1 full node (96 cores) achieves 956.17 **GFLOPS** (99.4% of the 962.0 GFLOPS 4-cluster node Roofline).

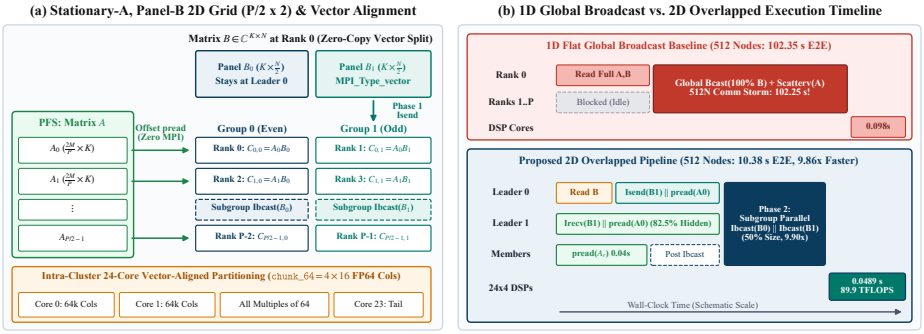


Fig. 3. Proposed Stationary-A, Panel-Broadcast-B 2D hybrid orchestration ($P/2 \times 2$): (a) zero-copy strided `MPI_Type_vector` dispatch of B panels, parallel `pread` of A, disjoint $C_{r,g}$ ownership, and 64-column partitioning; (b) asynchronous execution timeline contrasting 1D storm vs. 2D overlapped pipeline.

TABLE III
INTRA-NODE PERFORMANCE (GFLOPS), SPEEDUP, AND FP64 NUMERICAL ACCURACY VS. MULTI-THREADED ARMV8 OPENBLAS

	Single-Core Compute Density (1 Core)				Cluster-Level Throughput (1 DSP Cluster [24C] vs. CPU)					FP64 Max Error (ℓ_∞)	
Scale N ($M=N=K$)	DSP 1C **Ours**	CPU 1C 4xDGEMM	CPU 1C 1xZGEMM	Speedup (4D / 1Z)	DSP 24C **(1 Clust.)**	CPU 4C 4xDGEMM	CPU 16C 1xZGEMM	vs. 4C **(Equiv)**	vs. 16C (1Z)	$\max\lvert\Delta C_{\mathrm{Re}}\rvert$ (Real Part)	$\max\lvert\Delta C_{\mathrm{Im}}\rvert$ (Imag Part)
512	**44.16**	7.22	7.10	6.12× / 6.22×	**80.98**	23.32	40.99	3.47×	1.98×	7.28×10^{-12}	1.13×10^{-10}
1,024	**45.77**	7.07	7.69	6.48× / 5.95×	**139.73**	28.38	82.29	4.92×	1.70×	1.73×10^{-11}	3.27×10^{-10}
2,048	**49.19**	6.80	7.63	7.24× / 6.45×	**145.96**	22.84	92.49	6.39×	1.58×	4.37×10^{-11}	9.75×10^{-10}
4,096	**48.39**	6.78	7.74	7.13× / 6.25×	**173.05**	24.41	85.51	7.09×	2.02×	7.46×10^{-11}	2.79×10^{-9}
8,192	**49.16**	6.76	7.60	**7.28×** / **6.47×**	**243.44**	26.08	89.65	**9.33×**	**2.72×**	1.89×10^{-10}	7.57×10^{-9}
16,384†	**49.16**	7.20	OOM	6.83× / —	**199.21**	26.62	OOM	7.48×	—	1.05×10^{-9}	2.18×10^{-8}

†At $N = 16384$, allocating 12 GiB AoS buffers alongside planar host buffers exceeds 32 GB node RAM; DSP evaluation executes via 2×2 host-tiled scheduling.

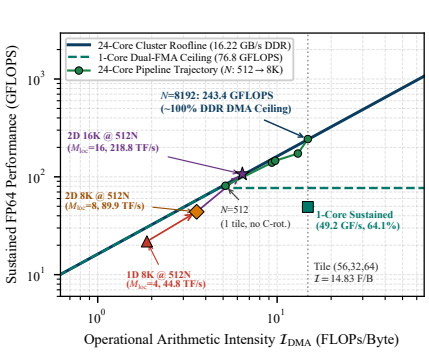


Fig. 4. Empirical cluster Roofline model (16.22 GB/s DDR4 DMA ceiling) and 512-node trajectory.

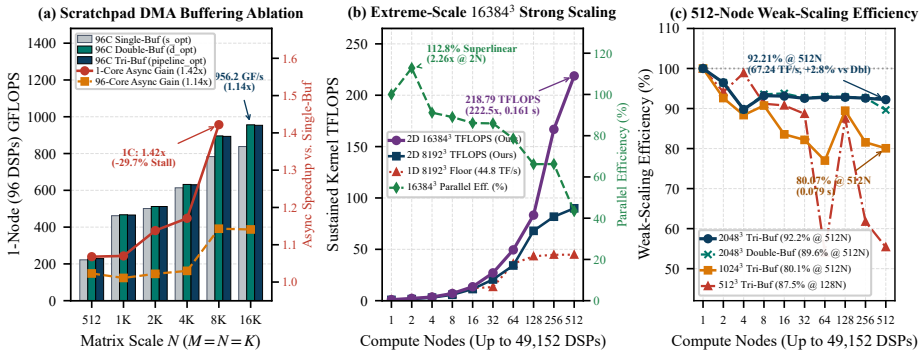


Fig. 5. Experimental scaling results: (a) DMA buffering ablation; (b) extreme 16384^3 strong scaling to 218.79 TFLOPS; (c) 1–512 node weak scaling maintaining 92.21% efficiency across 49,152 DSP cores.

- **DMA Buffering Ablation:** As shown in Fig. 5(a), asynchronous double buffering raises 1-core throughput from 33.79 to 48.05 GFLOPS (1.422× speedup, 29.69% runtime cut), and boosts 96-core node throughput from 838.03 to 956.17 GFLOPS (1.141×).
- **FP64 Precision:** Across all scales, element-wise maximum error against OpenBLAS scales as $\mathcal{O}(K\epsilon_{\mathrm{FP64}})$ bounded within 7.28×10^{-12} to 2.18×10^{-8} (Table III), confirming fused FMA stability.

B. Multi-Scale Distributed Strong and Weak Scaling

Table IV and Fig. 5(b)(c) report 1- to 512-node results:

- 8192^3 **Strong Scaling:** At 512 nodes, 1D stalls at the DDR streaming floor (44.78 TFLOPS) while broadcast latency explodes to 102.25 s. Our 2D co-design restricts communication to 10.32 s (9.90× **speedup**) and doubles kernel throughput to 89.92 **TFLOPS** (2.01× **speedup**), validating (7). Concurrent `pread` accelerates member I/O by up to 93.5× and hides 82.5% of leader I/O.
- **Extreme** 16384^3 **Strong Scaling:** Workload scaling (35.184 TFLOPs) achieves 112.80% **superlinear efficiency** at 2 nodes by halving cache/memory pressure, sustains 66.26% efficiency at 256 nodes (166.82 TFLOPS), and delivers 218.79 **TFLOPS** (0.1608 s, 222.49× speedup) at 512 nodes (49,152 DSPs), closely matching our theoretical prediction of 212.3 TFLOPS.
- **512-Node Weak Scaling:** Under 2048^3 base workload (68.72 GFLOPs/node), runtime rises by just 40.7 ms across a 512× scale-out (0.4825 → 0.5233 s), maintaining 92.21% **efficiency** (67.24 **TFLOPS**) at 49,152 DSP cores.

TABLE IV
MULTI-SCALE DISTRIBUTED STRONG AND WEAK SCALING ACROSS 1 TO 512 MT-3000 NODES (4 TO 2,048 MPI RANKS; 96 TO 49,152 DSP CORES)

			8192^3 **Strong Scaling** (4.398 **TFLOPs**)							**Extreme** 16384^3 **2D Strong** (35.18 **TF**)		
Nodes	**MPI Ranks**	**DSP Cores**	**1D Kern. (s)**	**1D TF/s (Eff. %)**	**1D Comm (s)**	**2D Kern. (s)**	**2D TF/s (Eff. %)**	**2D Comm (s)**	**Gain (Kern / Comm)**	**16K Kern. (s)**	**16K TF/s (Eff. %)**	**Weak TF/s (2048^3, Eff. %)**
1	4	96	4.5354	0.97 (100%)	9.01	5.7812	0.76 (100%)	2.89	0.78× / 3.12×	35.7795	0.98 (100%)	0.14 (100%)
2	8	192	2.3148	1.90 (97.9%)	13.94	2.8848	1.52 (100%)	5.09	0.80× / 2.74×	15.8621	2.22 (**112.8%**)	0.28 (96.5%)
4	16	384	1.2773	3.44 (88.7%)	18.02	1.4777	2.98 (97.8%)	7.10	0.86× / 2.54×	9.8105	3.59 (91.2%)	0.51 (89.8%)
8	32	768	0.6532	6.73 (86.7%)	18.88	0.7639	5.76 (94.6%)	7.70	0.86× / 2.45×	5.0231	7.01 (89.0%)	1.06 (93.2%)
16	64	1,536	0.3629	12.12 (78.1%)	20.28	0.3877	11.34 (93.2%)	9.73	0.94× / 2.08×	2.5957	13.56 (86.2%)	2.12 (93.1%)
32	128	3,072	0.3319	13.25 (42.7%)	24.26	0.2133	20.62 (84.7%)	8.33	**1.56**× / 2.91×	1.3007	27.05 (**86.0%**)	4.22 (92.6%)
64	256	6,144	0.1222	36.00 (58.0%)	32.21	0.1284	34.26 (70.4%)	10.26	0.95× / 3.14×	0.7112	49.47 (78.6%)	8.46 (92.8%)
128	512	12,288	0.1017	43.24 (34.8%)	26.96	0.0647	**68.01** (69.8%)	8.75	**1.57**× / 3.08×	0.4223	83.31 (66.2%)	16.92 (92.8%)
256	1,024	24,576	0.0984	44.70 (18.0%)	36.02	0.0538	**81.77** (42.0%)	10.43	**1.83**× / 3.45×	0.2109	**166.82** (66.3%)	33.76 (92.6%)
512	**2,048**	**49,152**	0.0982	44.78 (9.0%)	102.25	**0.0489**	**89.92** (23.1%)	**10.32**	**2.01**× / **9.90**×	**0.1608**	**218.79** (43.5%)	**67.24** (**92.2%**)

Triple buffering provides $+2.77\%$ higher throughput over double buffering by absorbing memory/NIC jitter.

VI. RELATED WORK

A. Scratchpad Co-Design and Software DMA Scheduling

Replacing hardware cache hierarchies with software-managed scratchpad memories (SPMs) eliminates cache tag overhead, coherence protocol traffic, and cache conflict misses, delivering higher energy efficiency and predictable execution [3]–[5]. Early pioneers such as the IBM Cell Broadband Engine [3] utilized memory flow controllers (MFC) to stage tiles across 256 KB local stores. The Sunway SW26010 architecture [5], [6] integrates 64 KB local data memory (LDM) per core, coordinating register communication buses for matrix multiplication. Modern spatial accelerators such as Tenstorrent, Cerebras CS-2, and Google TPU similarly rely on explicit SRAM tiling. Prior MT-3000 studies optimized real-valued GEMM, stencils, or attention mechanisms on single nodes [7]–[11], [16]. Our work differs fundamentally by addressing double-precision complex ZGEMM on an asymmetric dual-scratchpad architecture (64 KB SM + 768 KB AM), establishing a 14-channel tri-buffered DMA pipeline that hides bidirectional C-tile write-back latencies while pinning 32 vector registers for zero compiler spilling.

B. Dense Complex Linear Solvers in EDA Applications

Double-precision complex GEMM is the foundational computational kernel in large-scale electronic design automation (EDA), including full-wave electromagnetic boundary element methods (Method of Moments, MoM) for on-chip parasitic extraction, antenna array simulation, high-speed interconnect modeling, and quantum circuit state evolution [1], [2]. Standard open-source BLAS libraries such as OpenBLAS [17] and BLIS [2], [18] are heavily tuned for multi-level CPU cache hierarchies and rely on Array-of-Structures (AoS) layouts, which induce prohibitive vector lane shuffling overheads on wide 16-lane SIMD engines. Furthermore, algebraic 3M algorithms [2] reduce nominal FLOP counts but demand extensive workspace allocations and suffer $\mathcal{O}(\kappa)$ numerical cancellation [12]. Our planar split-complex 4M formulation avoids auxiliary memory while preserving full IEEE FP64 precision across all matrix dimensions.

C. Communication-Avoiding Distributed Matrix Multiplication

Distributed dense matrix multiplication has been extensively studied on cache-based homogeneous clusters. Canonical 2D SUMMA [14] and ScaLAPACK [19] broadcast sub-blocks in $\mathcal{O}(\sqrt{P})$ algorithmic steps, introducing fine-grained synchronization overheads. 2.5D GEMM [15] and 3D algorithms reduce communication by replicating input matrices c times, but require post-computation global reductions of the C matrix and exceed per-node memory budgets at large scale. Recent communication-optimal frameworks such as COSMA [20] and SLATE [21] optimize tile shapes for cache-coherent CPU/GPU systems but assume transparent hardware prefetching and symmetric interconnects. Our Stationary-A, Panel-Broadcast-B ($P/2 \times 2$) scheme completely avoids network communication of A and post-compute reductions of C, eliminates buffer packing via committed `MPI_Type_vector` strided datatypes, overlaps parallel POSIX `pread` with non-blocking P2P dispatch, and halves the off-chip DDR-streaming floor at extreme supercomputer scale.

VII. CONCLUSION

We presented an end-to-end hardware-software co-design framework for distributed double-precision complex GEMM on multi-zone heterogeneous DSP supercomputers. By coupling an intra-chip 14-channel asynchronous DMA tri-buffered pipeline and a 4×4 register-resident dual-FMA micro-kernel with an inter-node Stationary-A, Panel-Broadcast-B 2D hybrid orchestration, our solution breaks both the on-chip scratchpad memory wall and the inter-node network communication wall. Single-core throughput reaches 49.19 GFLOPS (64.1% peak), cluster throughput achieves 243.44 GFLOPS ($\sim 100\%$ Roofline saturation), and node throughput reaches 956.17 GFLOPS. Scaling to 512 MT-3000 nodes (2,048 MPI ranks, 49,152 DSP cores), our framework reduces collective communication time by $9.90\times$, doubles kernel throughput to 89.92 TFLOPS (8192^3), delivers 218.79 TFLOPS on extreme 16384^3 workloads ($222.49\times$ speedup, 112.80% 2-node superlinear efficiency), and sustains 92.21% weak-scaling efficiency. This co-design establishes a viable blueprint for deploying memory- and communication-intensive dense linear algebra solvers on next-generation heterogeneous accelerator architectures.

REFERENCES

[1] R. F. Barrett, T. H. Chan, E. F. D'Azevedo, and E. C. Jaeger, "Complex version of high performance computing LINPACK benchmark (HPL)," *Concurrency Computat.: Pract. Exper.*, vol. 22, no. 5, pp. 573–587, Apr. 2010.

[2] F. G. Van Zee and T. M. Smith, "Implementing high-performance complex matrix multiplication via the 3M and 4M methods," *ACM Trans. Math. Softw.*, vol. 44, no. 1, pp. 7:1–7:36, Jul. 2017.

[3] J. A. Kahle, M. N. Day, H. P. Hofstee, C. R. Johns, T. R. Maeurer, and D. Shippy, "Introduction to the Cell multiprocessor," *IBM J. Res. Dev.*, vol. 49, no. 4/5, pp. 589–604, Jul. 2005.

[4] F. D. Igual, M. Ali, A. Friedmann, E. Stotzer, T. Wentz, and R. A. van de Geijn, "Unleashing the high-performance and low-power of multi-core DSPs for general-purpose HPC," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2012, pp. 1–11.

[5] H. Fu, J. Liao, J. Yang, L. Wang, Z. Song, X. Huang, C. Yang, W. Xue, F. Liu, F. Qiao, W. Zhao, X. Yin, C. Hou, C. Zhang, W. Ge, J. Zhang, Y. Wang, C. Zhou, and G. Yang, "The Sunway TaihuLight supercomputer: System and applications," *Sci. China Inf. Sci.*, vol. 59, no. 7, p. 072001, Jul. 2016.

[6] L. Jiang, C. Yang, Y. Ao, W. Yin, W. Ma, Q. Sun, and F. Liu, "Towards highly efficient DGEMM on the emerging SW26010 many-core processor," in *Proc. 46th Int. Conf. Parallel Process. (ICPP)*, Aug. 2017, pp. 422–431.

[7] K. Lu, Y. Wang, Y. Guo, C. Huang, S. Liu, R. Wang, J. Fang, T. Tang, Z. Chen, B. Liu, Z. Liu, Y. Lei, and H. Sun, "MT-3000: A heterogeneous multi-zone processor for HPC," *CCF Trans. High Perform. Comput.*, vol. 4, no. 2, pp. 150–164, 2022.

[8] J. Fang, K. Yu, P. Zhang, D. Dong, X. Qi, X. Hou, R. Wang, and K. Lu, "mtGEMM: An efficient GEMM library for modern multi-core DSPs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 37, no. 4, pp. 905–919, Apr. 2026.

[9] X. Qi, J. Fang, P. Zhang, Y. Che, and J. Ren, "Constraint-driven auto-tuning of GEMM-like operators for MT-3000 many-core processor," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2025, pp. 185–199.

[10] H. Mo, Q. Wang, L. Liao, B. Li, L. Chi, and J. Liu, "Detailed analysis and optimization of irregular-shaped matrix multiplication on multi-core DSPs," in *Proc. 53rd Int. Conf. Parallel Process. (ICPP)*, Aug. 2024, pp. 719–729.

[11] X. Qi, J. Fang, P. Zhang, and Y. Che, "Optimizing attention for large language model inference on the MT-3000 many-core processor," *ACM Trans. Archit. Code Optim.*, vol. 23, no. 2, pp. 52:1–52:27, 2026.

[12] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[13] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[14] R. A. van de Geijn and J. Watts, "SUMMA: Scalable universal matrix multiplication algorithm," *Concurrency: Pract. Exper.*, vol. 9, no. 4, pp. 255–274, Apr. 1997.

[15] E. Solomonik and J. Demmel, "Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms," in *Proc. 17th Int. Conf. Parallel Process. (Euro-Par), Part II*, Aug. 2011, pp. 90–109.

[16] J. Li, H. Gu, J. Zhao, L. Qiao, C. Gong, and G. Zheng, "Transplantation and optimization of molecular dynamics simulation on MT-3000," *Future Gener. Comput. Syst.*, vol. 153, pp. 262–275, Apr. 2024.

[17] X. Zhang, Q. Wang, and Y. Zhang, "Model-driven level 3 BLAS performance optimization on Loongson 3A processor," in *Proc. IEEE 18th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2012, pp. 684–691.

[18] F. G. Van Zee and R. A. van de Geijn, "BLIS: A framework for rapidly instantiating BLAS functionality," *ACM Trans. Math. Softw.*, vol. 41, no. 3, pp. 14:1–14:33, Jun. 2015.

[19] J. Choi, J. Demmel, I. Dhillon, J. Dongarra, S. Ostrouchov, A. Petitet, K. Stanley, D. Walker, and R. C. Whaley, "ScaLAPACK: A portable linear algebra library for distributed memory computers—design issues and performance," *Comput. Phys. Commun.*, vol. 97, no. 1–2, pp. 1–15, Aug. 1996.

[20] G. Kwasniewski, M. Kabić, M. Besta, J. VandeVondele, R. Solcà, and T. Hoefler, "Red-blue pebbling revisited: Near optimal parallel matrix-matrix multiplication," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2019, pp. 24:1–24:22.

[21] M. Gates, J. Kurzak, A. Charara, A. YarKhan, and J. Dongarra, "SLATE: Design of a modern distributed and accelerated linear algebra library," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2019, pp. 26:1–26:18.